

ОТЧЕТ ПО ПРОЕКТУ: МНОГОСЛОЙНЫЙ ПЕРСЕПТРОН

Выполнил: Быданов Марат Эдуардович

1 Цель проекта

Целью данного проекта является реализация искусственной нейронной сети (многослойного персептрона) и алгоритмов её обучения с нуля, без использования готовых библиотек для машинного обучения (таких как Keras или PyTorch). В ходе работы необходимо было реализовать алгоритмы прямого и обратного распространения ошибки, градиентный спуск, а также применить их для решения задачи бинарной классификации (диагностика рака молочной железы).

2 Предварительный анализ и обработка данных (EDA)

Для обучения был предоставлен датасет `data.csv`, описывающий характеристики ядер клеток, извлеченных с помощью тонкоигольной биопсии. В ходе первичного анализа данных было выявлено следующее:

1. **Масштаб признаков:** Признаки имеют существенно разные диапазоны значений. Например, площадь клетки (`area`) может достигать значений свыше 1000, в то время как показатели гладкости (`smoothness`) измеряются в тысячных долях (0.01 - 0.05).
2. **Целевая переменная:** Столбец `diagnosis` содержит строковые значения 'M' (Malignant — злокачественная) и 'B' (Benign — доброкачественная).

Выводы из анализа и предобработка:

- Из-за сильного разброса значений было принято решение **обязательно стандартизировать данные** (Z-score normalization) по формуле:

$$X = \frac{X - \mu}{\sigma} \quad (1)$$

Без этого шага алгоритм градиентного спуска сходил бы крайне медленно, а функция активации (сигмоида) быстро уходила бы в насыщение (паралич сети).

- Для целевой переменной был применен **One-Hot Encoding** ('M' → [1, 0], 'B' → [0, 1]), так как на выходном слое сети требуется получить распределение вероятностей по двум классам с использованием функции Softmax.
- Для предотвращения переобучения и объективной проверки обобщающей способности модели, данные были разделены на **обучающую (80%)** и **валидационную (20%)** выборки с использованием фиксированного параметра `seed=42` для воспроизводимости.

3 Архитектура нейронной сети

Разработанный персептрон имеет модульную архитектуру, позволяющую задавать количество и размер слоев через аргументы командной строки. По умолчанию используется следующая конфигурация:

- **Входной слой:** 30 нейронов (соответствует количеству признаков).
- **Скрытые слои:** Два скрытых слоя по 24 нейрона.
- **Выходной слой:** 2 нейрона (по количеству классов).

Функции активации:

- Для скрытых слоев используется функция **Сигмоида**.

$$f(z) = \frac{1}{1 + e^{-z}} \quad (2)$$

Производная сигмоиды:

$$f'(z) = f(z) \cdot (1 - f(z)) \quad (3)$$

- Для выходного слоя применяется функция **Softmax**, чтобы получить на выходе распределение вероятностей, сумма которых равна единице.

$$Softmax(z_i) = \frac{e^{z_i}}{\sum e^{z_j}} \quad (4)$$

4 Алгоритмы обучения и оценка модели

Обучение сети происходит с использованием алгоритма **прямого и обратного распространения ошибки** (Backpropagation). Для оценки ошибки (loss) на каждой эпохе применяется функция **Бинарной кросс-энтропии**:

$$L = -\frac{1}{N} \sum_{i=1}^N (y_i \cdot \log(\hat{y}_i)) \quad (5)$$

Где y_i — истинная метка класса, а $\hat{y}_i$ — предсказанная вероятность.

Обновление весов происходит с помощью **мини-пакетного градиентного спуска** (Mini-batch Gradient Descent) со следующими гиперпараметрами:

- `learning_rate` = 0.0314
- `batch_size` = 8
- `epochs` = 84

5 Результаты работы

Программный комплекс разделен на три независимых модуля: `split_data.py`, `train.py` и `predict.py`. При запуске обучения на 84 эпохах наблюдалось стабильное падение функции потерь.

Анализ графиков кривых обучения: Как видно из графиков, метрики `loss` и `val_loss` стабильно снижаются и выполаживаются к концу обучения, достигая значения около 0.05. Точность на обучающей и валидационной выборках растет синхронно, без

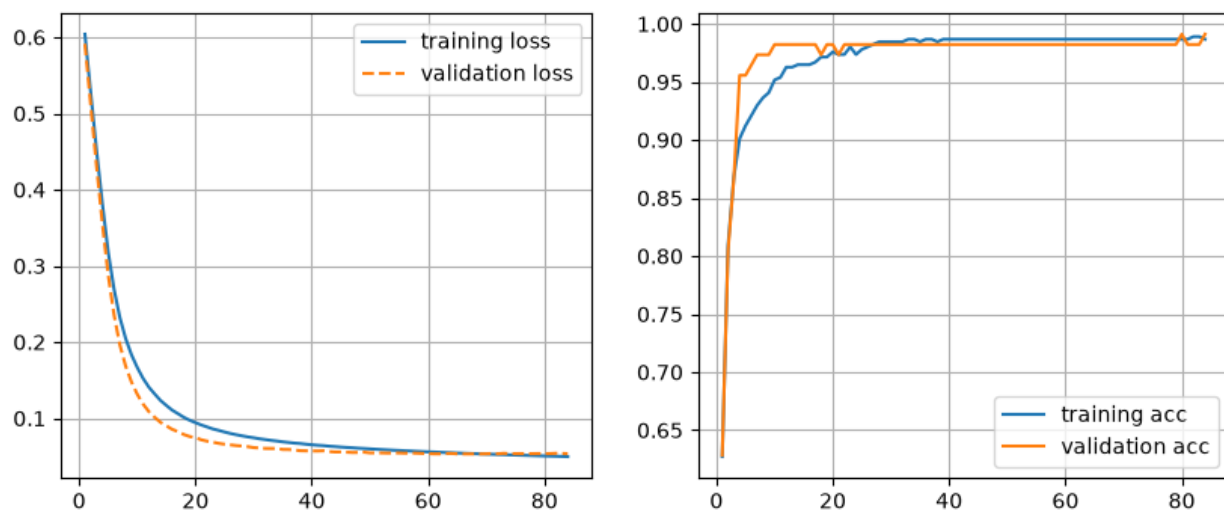


Рис. 1: Кривые обучения (Функция потерь и Точность)

резких скачков. Отсутствие значительного расхождения между линиями обучения и валидации свидетельствует о том, что модель успешно выявила скрытые закономерности в медицинских данных и **не переобучилась**.

При тестировании сохраненной модели (`saved_model.npy`) программой `predict.py` на отложенной валидационной выборке были получены следующие результаты:

- **Binary Cross-Entropy Loss:** 0.0523
- **Accuracy:** 98.23%

6 Вывод

В ходе выполнения проекта была успешно с нуля реализована архитектура многослойного персептрона. Самостоятельная реализация алгоритмов прямого и обратного распространения ошибки, а также математических функций активации и потерь, позволила на практике закрепить понимание внутренних механизмов работы искусственных нейронных сетей. Достигнутая точность классификации (98.23%) доказывает правильность написанного математического аппарата и успешность обучения модели.

Приложение: Основной исходный код (network.py)

В данном листинге представлена математическая реализация градиентного спуска, функций активации и потерь (Backpropagation) с нуля с использованием библиотеки NumPy.

```
1 import numpy as np
2 import pickle
3
4 def sigmoid(z):
5     z = np.clip(z, -500, 500)
6     return 1 / (1 + np.exp(-z))
7
8 def deriv_sigm(a):
9     return a * (1 - a)
10
11 def my_softmax(z):
12     z_exp = np.exp(z - np.max(z, axis=1, keepdims=True))
13     return z_exp / np.sum(z_exp, axis=1, keepdims=True)
14
15 def calc_loss(y_true, y_pred):
16     eps = 1e-15
17     y_pred = np.clip(y_pred, eps, 1 - eps)
18     return -np.mean(np.sum(y_true * np.log(y_pred), axis=1))
19
20 def calc_acc(y_true, y_pred):
21     return np.mean(np.argmax(y_true, axis=1) == np.argmax(y_pred, axis=1))
22
23 class MLP:
24     def __init__(self, layers, lr=0.01):
25         self.lr = lr
26         self.num = len(layers)
27         self.W = []
28         self.B = []
29
30         for i in range(self.num - 1):
31             w = np.random.randn(layers[i], layers[i+1]) * np.sqrt(2 / layers[i])
32             b = np.zeros((1, layers[i+1]))
33             self.W.append(w)
34             self.B.append(b)
35
36     def forward(self, x):
37         self.A = [x]
38         tmp = x
39         for i in range(self.num - 2):
40             z = np.dot(tmp, self.W[i]) + self.B[i]
41             tmp = sigmoid(z)
42             self.A.append(tmp)
43
44         z_out = np.dot(tmp, self.W[-1]) + self.B[-1]
45         out = my_softmax(z_out)
46         self.A.append(out)
47         return out
48
49     def backward(self, x, y):
50         m = x.shape[0]
51         pred = self.A[-1]
52         dz = pred - y
53
54         for i in range(self.num - 2, -1, -1):
55             a_prev = self.A[i]
56             dw = np.dot(a_prev.T, dz) / m
57             db = np.sum(dz, axis=0, keepdims=True) / m
58
59             if i > 0:
60                 da = np.dot(dz, self.W[i].T)
61                 dz = da * deriv_sigm(self.A[i])
62
63             self.W[i] -= self.lr * dw
64             self.B[i] -= self.lr * db
```

Листинг 1: Реализация ядра нейронной сети